

AI & Analytics — Data Collection Inventory

Scope: every piece of student/teacher data that feeds EEC's AI features (AI tutor, RAG retrieval, mastery/recommendation engine, reading/writing assessment, engagement scoring) and the analytics/dashboards built on top of it. This is narrower than the full platform data inventory (EEC_Data_Collection_Purpose_and_Use_Inventory.pdf, which also covers fees, HR, attendance, etc.) — it's the AI/ML/analytics slice only.

Grounded directly in the current codebase (backend Mongoose models, backend/services/, ai-service/app/*) — not aspirational. Prepared 2026-08-26.*

1. AI Tutor — Conversations & Memory

Models: `backend/models/TutorConversation.js`, `backend/models/StudentMemorySummary.js`

Field	Notes
<code>studentId</code> , <code>schoolId</code> , <code>campusId</code>	required identifiers
<code>clientId</code>	frontend-generated id, lets client upsert idempotently
<code>title</code> , <code>subjectTitle</code> , <code>topicTitle</code>	chat metadata
<code>messages[].role</code> (<code>user/assistant</code>), <code>.text</code> , <code>.error</code>	full verbatim text of every prompt and every AI response
<code>summary</code> (<code>StudentMemorySummary</code>)	rolling LLM-generated summary of past sessions
<code>keyInsights[]</code>	AI-extracted bullet facts about the student's learning patterns/style
<code>sessionCount</code> , <code>lastSummarizedAt</code>	

Why: answer learning questions, preserve conversation continuity, personalise future tutoring sessions.

Use: every chat turn is stored verbatim in `TutorConversation`. After enough turns, `POST /generate/summarize-session` (ai-service) condenses the conversation into `summary` + `keyInsights`, persisted to `StudentMemorySummary`. Both the profile (\$6) and the rolling summary are injected into every subsequent tutor prompt via `studentContextBuilder.js` so the AI "remembers" the student without the student re-explaining themselves.

Compliance flag: neither model has field-level encryption — `messages[].text`, `summary`, and `keyInsights` are plaintext in MongoDB (unlike `StudentUser/ParentUser`, which encrypt contact fields, and `ChatMessage`, which supports end-to-end encryption for human-to-human chat).

External processing: by default all generation runs on local Ollama models. If `OPENROUTER_API_KEY` is set in `ai-service/.env`, every `/generate/*` call (including tutor chat) switches to OpenRouter — meaning prompt text and retrieved material would leave the local infrastructure

to an external model provider. Currently unset in this deployment's `.env` at time of writing, but the switch is a single config value, not a code change.

2. Teaching Material Retrieval (RAG)

What's ingested: teacher-uploaded PDFs/docs/slides → parsed (PyMuPDF / Tesseract OCR / python-docx/pptx) → chunked → embedded (`nomic-embed-text`) → upserted into Qdrant collection `teacher_documents`.

Qdrant payload fields per chunk: `school_id`, `class_id`, `section_id`, `subject_name`, `chapter_title`, `topic_title`, `material_id`, `start_char`, `chunk_type` (`text/visual`), `page_number` (visual chunks), `source_url`, `source_name`, `discipline`, `curriculum_code`.

Why: ground every AI tutor answer, quiz, and diagram in institution-approved material only — never open-ended model knowledge.

Use: student questions are embedded and matched against this collection (chapter-scoped first, subject-wide fallback) with a relevance threshold (`rag_relevance_threshold = 0.55`). Retrieved chunks are stripped of "Note to Teacher" facilitator content (`_strip_teacher_notes`) before ever reaching student-facing output. Deleting a material also fires a delete against the matching Qdrant vectors (`DELETE /ingest/material/{id}`).

3. Vision-Model Page Extraction (llava:13b)

Ingestion time (`app/modules/vision/client.py`): every teacher-uploaded page image is read by `llava:13b`, extracting a structured JSON object:

`visible_text`, `formulas`, `units`, `diagram_labels`, `chart_labels`, `description`, `uncertainties`.

This is stored as a `chunk_type: "visual"` entry in Qdrant alongside the text chunks — not student data, but teaching-material content that AI now "sees."

Generation time (new, added today): for `visual_explain` mode specifically, when retrieval has already surfaced a cited visual page, the backend now re-renders that exact PDF page and asks `llava:13b` a fresh, question-specific query about it (`_live_visual_grounding()` in `ai-service/app/modules/chat/service.py`) — so the diagram/explanation the student sees is grounded in a live read of the page for their specific question, not just the generic one-shot ingestion-time description. This adds one extra `llava:13b` call (image + question) per applicable tutor request; it fails soft (falls back to the ingestion-time description) if the vision model is unavailable.

Why: let the AI tutor read and explain diagrams, charts, and formulas from scanned/complex textbook pages, not just plain text.

4. Mastery, Progress & Spaced Repetition

Models: `MasteryScore.js`, `FlashcardResult.js`, `SpacedRepetition.js`, `SpacedRepetitionSchedule.js`

Model	Key fields
MasteryScore	studentId, schoolId, subject, topicId, topicTitle, chapterTitle, score (0–100), attemptCount
SpacedRepetition	per-flashcard SM-2 state: cardHash, cardFront, cardBack (verbatim flashcard text), interval, easeFactor, repetitions, nextReview, lastReview
SpacedRepetitionSchedule	per-topic SM-2 state: subject, topicTitle, stage (0–4), intervalDays, lastScore, nextReviewDate

Why: track what a student has mastered vs. is still learning, and schedule review at the interval that maximises retention (1→3→7→14→30-day SM-2 curve).

Use: `engagementScorer.js` reads **SpacedRepetitionSchedule** for overdue items and sends review-nudge notifications. `developmentProfileService.js` blends **MasteryScore** (cognitive category) and **FlashcardResult+SpacedRepetitionSchedule** (memory category) into the 6-category development profile (§6). None of these four models encrypt any field.

5. Recommendations & Gap Detection

Model: `StudentInsight.js` — studentId, schoolId, insightType (gap_detection/mastery_milestone/at_risk/improvement), subject, title, summary, payload (free-form Mixed), seenByTeacher.

Data flow:

- `gapDetectionEngine.js` reads every **MasteryScore** for a student+subject plus the subject's **CurriculumMap** ordering, flags topics below a 60% mastery threshold, and walks backward to identify likely unmastered prerequisite "root causes." When ≥ 2 root causes are found, it **persists a `StudentInsight`** and **notifies the student's teacher(s)** by name, naming the specific weak topics.
- `recommendationEngine.js` reads **MasteryScore**, **CurriculumMap**, **SpacedRepetitionSchedule**, and the latest gap-detection **StudentInsight** to produce one "what to study next" recommendation with an explainability string.

Why: surface actionable, explainable next steps to students and early-warning signals to teachers, instead of a black-box score.

Compliance flag: this is the mechanism that exposes a student's specific academic weak points directly to teacher notifications — worth confirming visibility/retention rules match the platform's sensitive-data handling expectations (see row 13, "Behaviour, wellbeing" caveat, in the platform-wide inventory).

6. Holistic Development Profile (6 categories)

Model: `StudentDevelopmentProfile.js` — one subdocument per category (`cognitive`, `memory`, `creative`, `language`, `socialEmotional`, `physical`), each holding `score` (0–100 or `null`), `trend` (`improving/stable/declining/unknown`), `lastUpdated`, `dataPoints`.

Computed by ``developmentProfileService.js`` from:

- Cognitive ← `MasteryScore`
- Memory ← `FlashcardResult` + `SpacedRepetitionSchedule`
- Language ← `ReadingAssessment` + `WritingAssessment`
- Social-Emotional ← `Wellbeing` (mood, social engagement) + `StudentObservation` (teacher-recorded mood ratings, last 10)
- Creative, Physical ← manual/offline teacher input only (no automated signal yet)

Why: give the AI tutor (and Bloom's-taxonomy targeting) a directional sense of a student's strengths/weaknesses beyond raw quiz scores, and give parents/teachers a plain-language 3-section view (Academic Growth / Emotional Wellbeing / Overall Mastery).

Use: `formatProfileForLLMO` serialises this profile into a plaintext block **injected directly into every AI tutor system prompt** so responses are calibrated to the student without ever stating the scores back to them. `getBloomRecommendation()` derives which Bloom's-taxonomy level (remember → create) the tutor should target based on the weakest category.

7. Reading & Writing Assessment

Models: `ReadingAssessment.js`, `WritingAssessment.js`

Model	Sensitive fields
<code>ReadingAssessment</code>	<code>transcript</code> (Whisper speech-to-text of the student's spoken reading, plaintext), <code>audioDurationSeconds</code> , <code>scores</code> . { <code>overall</code> , <code>pronunciation</code> , <code>grammar</code> , <code>fluency</code> , <code>confidence</code> , <code>accent</code> , <code>reading_speed</code> }, <code>mispronounced_words[]</code> , <code>missed_words[]</code> , <code>extra_words[]</code> , <code>rawEvaluation</code> (Mixed)
<code>WritingAssessment</code>	<code>submission</code> (student's full verbatim written text, plaintext), <code>scores</code> . { <code>overall</code> , <code>grammar</code> , <code>vocabulary</code> , <code>tone</code> , <code>coherence</code> , <code>verb_tense</code> , <code>sentence_structure</code> , <code>creativity</code> }, <code>corrections[]</code> , <code>improvedVersion</code> , <code>cefrLevel</code> , <code>rawEvaluation</code> (Mixed)

Why: give language-skill feedback and feed the Language category of the development profile.

Use: audio is transcribed locally via Whisper (`whisper_model_size` = `large-v3-turbo`, GPU/`float16`); a dedicated local model (`ollama_assess_model` = `qwen3:8b`) scores pronunciation/grammar/fluency etc. Both models set an `embeddingStored` boolean flag, and config defines a separate Qdrant collection (`qdrant_language_collection` = `student_language_memory`) — consistent with assessment content also

being embedded for adaptive language-memory retrieval, though the exact write path wasn't traced in this pass.

Compliance flags:

- Neither model encrypts any field — raw speech transcripts and full writing submissions are plaintext.
- Whether raw **audio** itself (not just the transcript) is retained, and for how long, is not confirmed in this pass — worth an explicit check before this goes into a privacy policy.
- Unlike every other AI/analytics model in this inventory, ``schoolId`/`campusId`` **are not `required`** on either model (only `studentId` and the material/prompt reference are mandatory) — an inconsistency worth fixing for tenant-isolation guarantees.

8. Engagement Scoring & Badges

``engagementScorer.js`` (no dedicated model — computed on read, not persisted):

- Reads `TeachingMaterial.engagement` (`viewCount`, `timeSpent` — school-wide, not per-student) and `PracticeAttempt` (`studentId`, `subjectId`, `isCorrect`, `createdAt`).
- Formula: `score = (timeSpentMin×0.4) + (quizAttempts×10×0.3) + (viewCount×5×0.3)`, normalised to 100, output per subject/topic with an `isLow` flag.

``StudentBadge.js`` — `studentId`, `schoolId`, `badgeType` (`mastery/streak/completion/challenge/engagement`), `title`, `subject`, `topicTitle`, `awardedAt`.

Why: surface disengagement early, and gamify/recognise progress (badges feed goals 2 & 11 from the Goals & Skills brainstorming doc).

9. Third-Party / External Processors Used by AI & Analytics

Processor	Role	Default
Ollama (local)	chat (<code>qwen3:8b</code>), embeddings (<code>nomic-embed-text</code>), summaries (<code>qwen2.5:7b</code>), vision (<code>llava:13b</code>), assessment (<code>qwen3:8b</code>)	local, on-prem/self-hosted
OpenRouter	drop-in replacement for all <code>/generate/*</code> calls when <code>OPENROUTER_API_KEY</code> is set	disabled unless configured
Qdrant	vector store for teaching-material embeddings (<code>teacher_documents</code>) and language-assessment embeddings (<code>student_language_memory</code>)	configurable (cloud or local)
Whisper (<code>large-v3-turbo</code>)	speech-to-text for reading assessment	local, GPU

Note: the actual live `.env` for this deployment currently resolves `ollama_model` to `qwen3:8b` (not `llama3.2:3b`, the code default) and `ollama_summary_model` to `qwen2.5:7b` (not `14b`) — there are also three duplicate `OLLAMA_MODEL=` lines in `ai-service/.env`, last one wins. Worth cleaning up so the deployed models match documentation.

10. Open Compliance Items (factual, not resolved)

1. **No field-level encryption on any AI/analytics model** — `TutorConversation.messages[].text`, `StudentMemorySummary.summary/.keyInsights`, `ReadingAssessment.transcript`, `WritingAssessment.submission` are all plaintext in MongoDB, unlike `StudentUser/ParentUser` contact fields.
2. ``ReadingAssessment`/`WritingAssessment`` don't require ``schoolId`/`campusId`` — every other AI/analytics model in this inventory enforces both.
3. **No confirmed end-to-end retention/deletion process** spanning Qdrant vector records, AI tutor memory (`TutorConversation`, `StudentMemorySummary`), and assessment audio — carried over from the platform-wide inventory (row 26) and directly relevant here since this is exactly the data most likely to be considered sensitive "AI-derived" data under most privacy frameworks.
4. **External LLM routing (OpenRouter) is a single config flag away from sending tutor prompts/material off-infrastructure** — should be listed in a subprocessor record with purpose/retention/training-use terms before being enabled in any production school tenant.
5. ``StudentInsight.payload`` (gap-detection root causes) is pushed into teacher notifications by name — confirm this matches the school's intended visibility rules for a "weak topic" signal about a specific student.

EEC — AI & Analytics Data Collection Inventory | Internal draft — confirm scope, retention and legal basis before external publication.